

// **WAYSCONF 2026 · MASTERCLASS · 90 MINUTES**

Build an AI that checks and fixes its own work

Agentic loops in practice.

Szymon Paluch · CTO, Susteen

ROOM-PM · 16 September 2026

// 0:00 – 0:08

The case

Most AI demos stop here

A prompt goes in. Something impressive comes out. Everyone claps.

Then someone asks a question the demo was not built to survive:

Does it match the design?

Is the contrast legal?

Does it work on a phone?

Is that the actual copy, or copy it invented?

no answer

This is a talk about the word *no*

A loop is only worth building if something in it can refuse
the work without asking a language model for an opinion.

What you will have at 16:25

A landing page for a music festival that does not exist, built from a Figma file by an agent you instructed, checked by a program that agent wrote and is not allowed to change, fixed when it failed, and deployed to a URL you can send to someone.

```
exit 0
```

YOU LEAVE WITH

A URL, live on the internet.

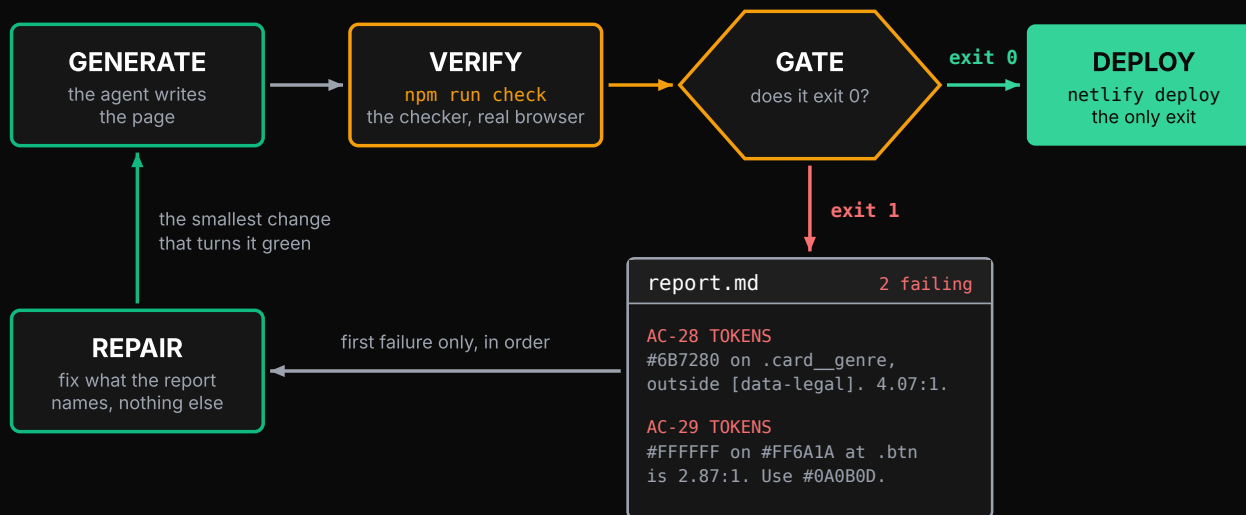
A folder: the page, and the checker that judged it.

Eight prompts that work on Monday, on something that is not a festival.

The loop

The loop

Four steps and one exit. The step that decides is a program, and it decides the same way every time.



The arrow back carries a file. Nothing on this path asks the model whether the work is good.

Setup

An agent running in an empty folder

You need two things

THE DESIGN

The TURBINE file in Figma, saved as one `.fig` that your agent reads by itself. No Figma account? The same file is on the page.

THE PROMPTS

waysconf.szymonpaluch.com

Eight of them, with a copy button on each.

AND AN EMPTY FOLDER

```
mkdir turbine && cd turbine
claude          # or: codex
```



There is **nothing to clone** and no code to read.

Agent up? Paste **prompt 01** now.

The terminal, in one slide

```
pwd          # where am I
ls           # what is here
cd turbine   # go in there
claude       # or: codex
```



Ctrl+C stops whatever is running.

It is a text field that runs programs.

It gives you no feedback while it works, which reads as "broken" and is almost always "busy".

When a command finishes you get the prompt back. When you do not get the prompt back, it is still going.

Full version: the Before tab, for your operating system

// 0:15 — 0:25

Design in, not screenshots in

Paste prompt 02 now, then listen

Put the `.fig` in a folder called `design`, next to your project, and paste **prompt 02**.

WHAT IT DOES

Your agent has never seen a `.fig`. It writes a decoder, reads what comes out, and tries again until twelve cards carry twelve names.

That is the first loop of the day, and you did not have to set it up.

WHAT YOU DO

Nothing, for ten to fifteen minutes. Let it use the network if it asks — it may install a small package.

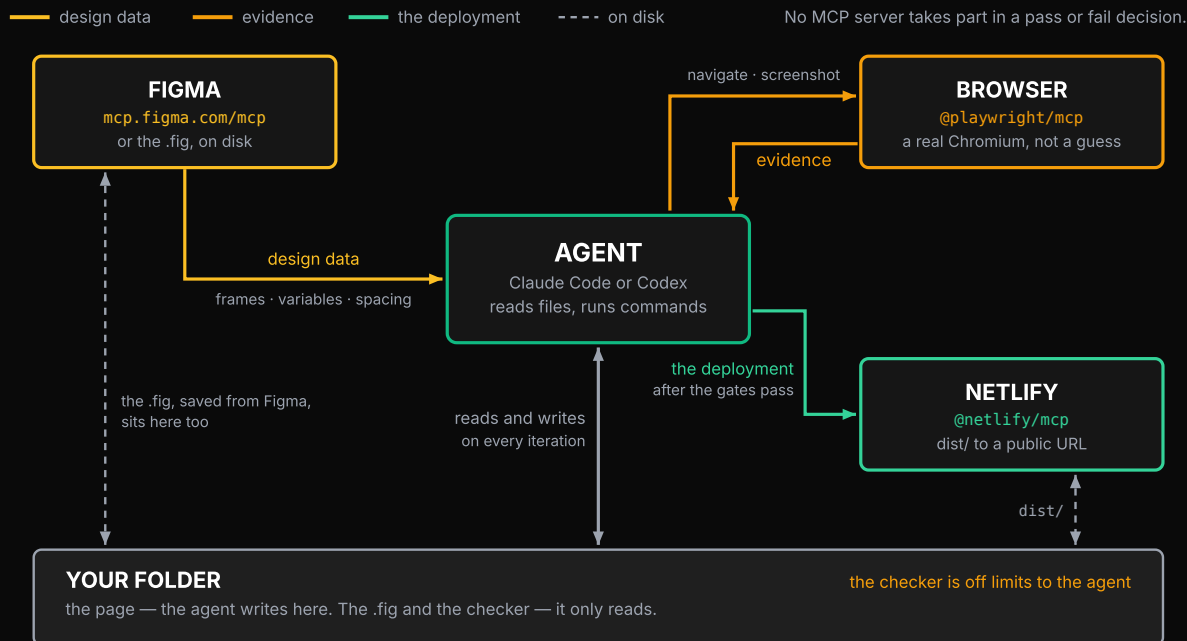
It writes no page. It will stop and tell you what it found.

MCP, in one sentence

A standard way for a coding agent to call tools that are not inside it.

What MCP actually connects

Three tools around one agent, and a folder underneath that every one of them reads and writes.



Two ways to hand over a design

A SCREENSHOT

The agent looks at pixels and infers.

The orange becomes *an* orange.

The spacing becomes *some* spacing.

It looks roughly right.

token gate fails

THE FILE ITSELF

Colours arrive as values. Spacing arrives as numbers.

```
TURBINE / Color
accent/sodium    #FF6A1A
text/secondary   #A7AEBB
text/muted       #6B7280
```



Nothing is guessed, so nothing is guessed wrong.

token gate passes

The honest bit about Figma

The official Figma MCP server needs a paid seat, and even then it runs on a budget. On a free account that budget is 20 calls a month — gone before the design is read.

SO YOU HAND OVER THE FILE

```
File → Save local copy → turbine.fig
  canvas.fig   the whole document
  images/      every photo
```



Nothing in this workshop is blocked by a free Figma account.

AND THE AGENT DECODES IT

It has never seen a `.fig`. It writes a decoder, reads what comes out, and tries again until twelve cards carry twelve names — the first loop of the day. It may install a package to do it, so let it use the network.

Plan first, then build

Read the plan before it writes the code

Plan mode is the cheapest correction in the entire loop. A wrong assumption costs one sentence to fix here, and twenty minutes of generated code to fix later.

```
# Claude Code: Shift+Tab until the footer says "plan mode on"  
# Codex CLI: ask for a plan and refuse to accept code until you have read it
```



Then read it. Actually read it. The plan is where you find out it misunderstood the brief.

Your turn — 15 minutes

1. READ WHAT 02 FOUND

Check two things against the design yourself.
Then answer its **point 6** — the things the file does not say — in your own words.

2. PASTE PROMPT 03

It builds one section, stops, and waits for you to say **next**.

02 still decoding? Let it finish. Nothing here is a race.

Falling behind cannot hurt you.

Stop where you are. Leave whatever is unfinished. I want to move on.

Every prompt stands alone. A half-built page with a working checker teaches more than a finished page with none.

The verifier

The only part that matters

Paste prompt o4 now, then listen

This is the workshop.

Your agent writes a program that judges its own page — and in the next prompt, you forbid it from ever touching that program again.

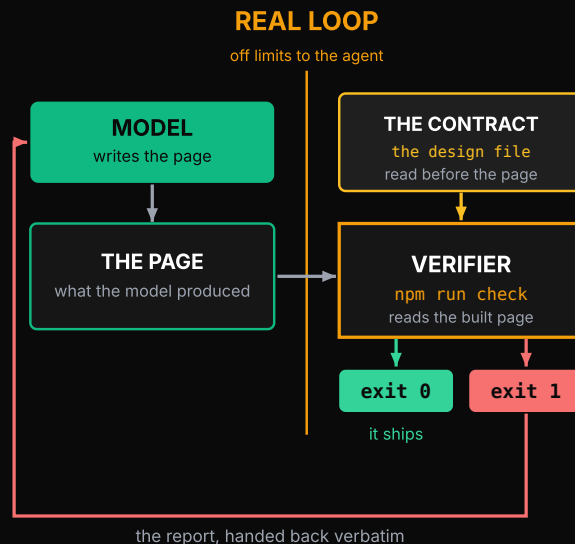
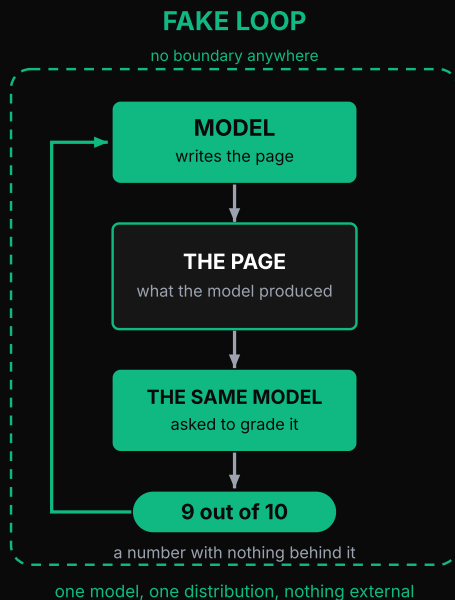
it should be red

About eight minutes. If it passes first time, it is not checking anything.

A real loop and a fake one

Fake loop, real loop

The same model writes the same page. What changes is who is allowed to say no.



Where self-assessment is fine, and where it is not

FINE

Is this sentence any good.

Which of these five findings matters most.

Does this read as the same brand.

Is the hierarchy clear.

Taste, prioritisation, judgement.

NOT FINE

Is this correct.

Is it accessible.

Does it match the design.

Is it finished.

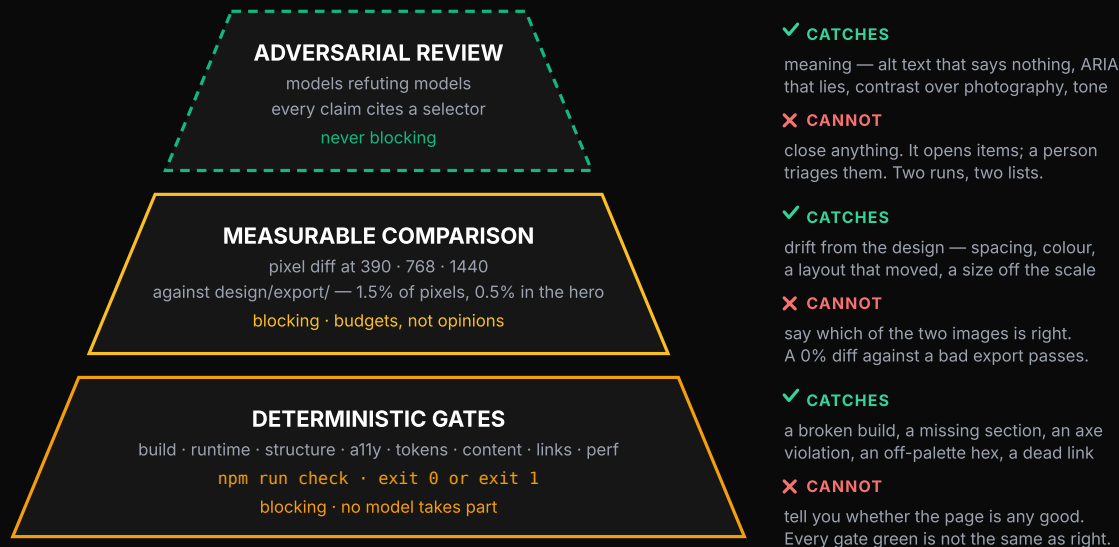
Anything with an external truth.

Use the external truth.

Three tiers of checking

Three tiers of verification

The bottom tier is the widest, the cheapest and the only kind that can be trusted to say no on its own.



The build gate runs first and alone; the seven browser gates then run together, and Lighthouse last.

Only the bottom two tiers can fail the build. Adversarial review runs after npm run check exits 0, and never blocks.

Automated accessibility rules reach roughly 30 to 40 per cent of WCAG — a widely cited estimate, not a measurement of this page.

The reference site: ten gates, no opinions

GATE	WHAT IT DECIDES	CRITERIA
Build	It compiles, no type errors, one page out	AC-01...05
Runtime	No console errors, no failed requests	AC-03...04
Structure	Sections, headings, landmarks, skip link, ids	AC-06...14
Accessibility	axe clean at three widths, focus, keyboard, motion	AC-15...25
Design tokens	Every colour and size came from the design	AC-26...33
Visual fidelity	Pixels match the design, within a stated budget	AC-34...37
Content	The real copy, all twelve artists, no placeholders	AC-38...47
Links	Nothing points at nothing	AC-48...51
Lighthouse	Performance, a11y, best practices, SEO, and five budgets	AC-52...56
Text over images	The contrast axe refuses to judge	AC-61

Watch it catch something real

The palette has a trap in it, on purpose.

```
color.text.muted    #6B7280  
color.bg.base       #0A0B0D
```



4.07 : 1

Under the 4.5:1 that body text requires.

```
color.text.secondary #A7AEBB  
color.bg.base        #0A0B0D
```



8.9 : 1

Ask any model for "a muted grey for supporting copy" and watch which one it reaches for.

The failure report is the interface

```
**3. [check 07] a colour on the page is not in the design**
```

- Where: the supporting line inside each card in the lineup section
- Expected: the design's "text / secondary"
- Actual: the design's "text / muted" – measured 4.07:1 against the page background,
and body text needs 4.5:1
- Hint: muted is defined for legal and footer text only



No model wrote that. A program measured it and printed it.

The agent's next iteration starts by reading this file.

The rule that holds it all up

The agent may never edit the verifier.

It is in prompt 05, in capitals, and it comes first:

```
1. NEVER change the checker to make a check pass. Not a threshold,  
   not a skipped assertion, not a rule switched off. If you believe  
   a check is wrong, STOP, tell me which one and why.
```



Here is what actually happened

These prompts, an **empty folder** outside every repository, nothing else.

PROMPT	AGENT	RESULT
01 — a project exists	Codex	107 s
02 — reads the <code>.fig</code>	Claude Code	16 min — 12/12 names, every colour, all widths
03 — a hero from the design	Codex	41 s
04 — writes its own checker	Codex	495 s — 264 lines, tests unasked
05 — the loop runs	Codex	46 s

Unasked, 04 treated axe's *incomplete* results as failures — from one sentence: **"a check that cannot run is a failure, never a skip"**.

And here is what went wrong

Thirteen incidents while building this. **Nine were failures of the verifier or the harness, not of the page.**

WHAT

Nine gates ran against **somebody else's website** for a whole run — 427 confident, correctly-formatted failures

axe passed a blank page. Zero violations, three breakpoints, green

The loop hung for an hour on a stdin nobody closed

The **evidence harness fabricated** four clean iterations out of a killed run

The gate was green and **the hero was illegible** — five of nine pieces of text below the legal minimum

WHY IT IS IN THE DECK

A verifier that is confident and wrong is worse than none

A measurement of nothing looks exactly like a measurement of perfection

Third unbounded wait in one project

Absence read as success — fourth time, and the only one that invented a table

axe does not fail text over a photograph. It marks it *incomplete*

// 0:55 — 1:10 • SPRINT 2 RUNS IN THE BACKGROUND

The loop, and workflows

Start your loop now, then listen

Paste **prompt 05**. Then leave it alone — I will talk over it, and when it stops we will look at what it did.

```
Run npm run check. If it exits 0, stop and tell me.  
If it does not, read the report it wrote, fix what it names,  
run it again. Repeat until it exits 0.  
  
NEVER change the checker to make a check pass.
```



On Claude Code you can make it structural instead of trusted:

```
/goal npm run check exits 0
```



The session cannot end until that command actually exits 0. Codex has no equivalent — there, the paragraph is the mechanism.

The whole loop is four sentences

Check first	Run the check <i>before</i> you fix anything. An agent asked to improve passing work will find a reason.
Hand back the report	Unchanged. The file a program wrote, not your summary of it.
Never touch the checker	The moment the thing being judged can edit the judge, every green result afterwards means nothing.
Stop on a program	Not when it feels finished. When something exits 0.

No script. Nothing to install. You can say this to any agent, today.

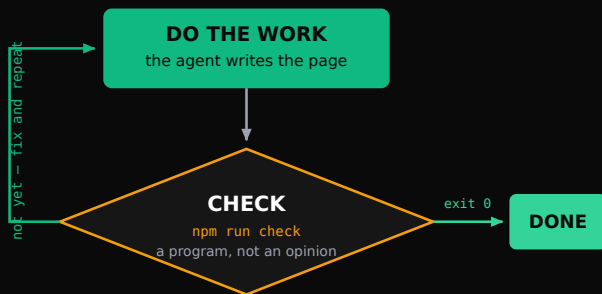
Two shapes, and neither is a script

Two shapes

Almost everything you will build on an agent is one of these. Neither is a script.

THE LOOP

do this again until a condition holds



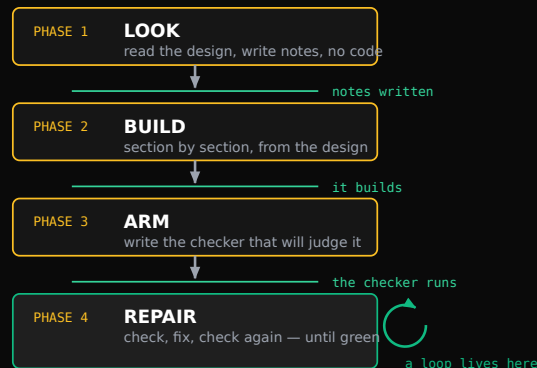
You define the **exit condition**.

"Run the check. Fix what it names. Repeat until it exits 0."

prompt 05 · /goal in Claude Code

THE WORKFLOW

do these, in order, with a bar between each



You define the **phases**, and what must be true before each one ends.

prompt 08

Neither of these is a script. Both are things you **say** — which is why changing one is a sentence, not an edit.

They nest: the workflow gets you to nearly-right, and a loop inside one phase closes the last gap.

Why a fresh context beats a long one

ONE LONG CONVERSATION

Forty minutes of history.

Three abandoned approaches still in context.

The original brief, sixty thousand tokens ago.

Reasoning gets worse as it fills.

A FRESH CONTEXT EACH PASS

Reads the report — the current failures.

Reads `notes.md` — what was already tried, and what it cost.

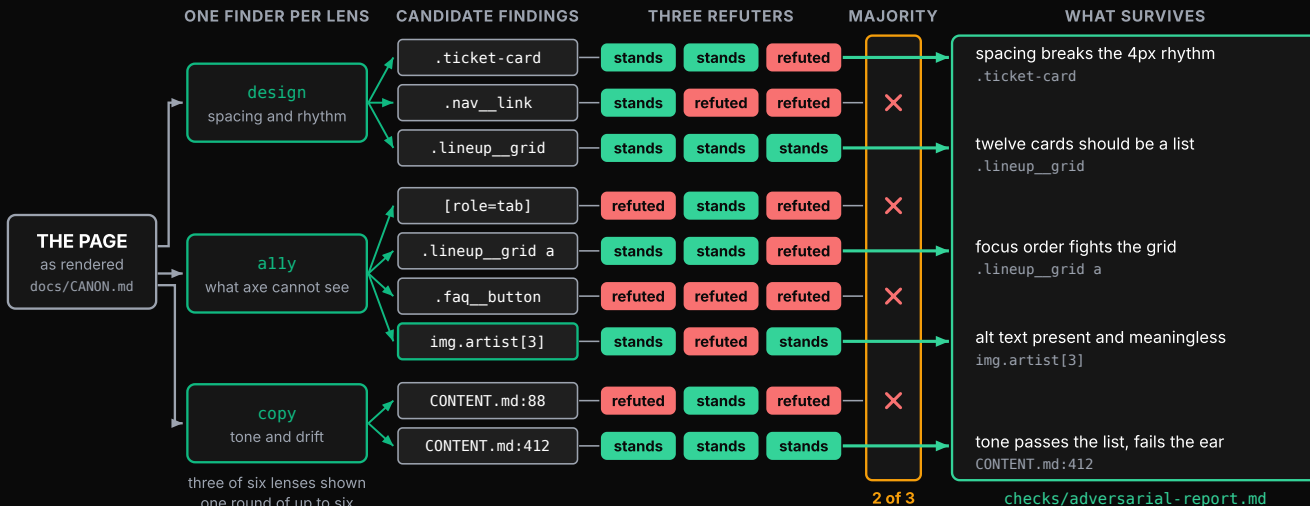
Reads the brief.

Notes on disk beat memory in context.

When one agent is not enough

One finder per lens, three refuters per finding

The work fans out, gets argued with, and only the part that survives a majority reaches a person.



Nothing here runs in lockstep.

Each candidate goes to three refuters whose only instruction is to argue that it is wrong, and reaches the filter when its own panel finishes. A lens that returns two findings does not hold up a lens that returns four.

The filter is arithmetic: two of three must stand. Every claim cites a selector or a file and line, or it is discarded unread.

Nothing on this page sets pass or fail. It opens items for a person to triage.

Adversarial review

The instruction is not "review this".

The instruction is **"refute this, and default to refuted if you are unsure"**.

```
ultracode. ... inside a phase, spawn as many subagents as the work  
needs and run them in parallel, deciding how many from what you find.
```

```
PHASE 6 – ATTACK
```

```
Send every candidate finding to a separate subagent whose only job  
is to argue that it is wrong. Report only what survives.
```



That is **prompt 08** — the whole run as one message. Take it home. Today, if there is time, **prompt 07** is the one-agent version.

Ship it

Deploy

Paste **prompt 06**. It builds, deploys to Netlify, and then checks that the page the server returns is the page that passed — not an older one.

It will ask you to sign in, then to pick or create a site. Take the URL it prints.

Put your URL on the board. We are going to look at all of them.

The gate was green. The hero was illegible.

Zero axe violations. Three breakpoints. Six page states. Twice, deterministically. Lighthouse accessibility **100**.

Then I measured the hero by hand — screenshot the page with every glyph made transparent, sample the brightest patch behind each line, compute the real ratio.

TEXT	MEASURED
"Fourth edition"	2.94:1
"Ambient, techno and modular..."	3.02:1
"The Powerhouse, Hall E"	4.25:1
"Friday 12 – Sunday 14 June"	4.41:1

WHY AXE SAID NOTHING

It does not evaluate text over a background image. It does not fail it — it marks the pair **incomplete**, and in a zero-violations gate that is indistinguishable from correct.

Reading the CSS would not have found it either. The background there is a photograph, two scrims and a gradient composited together. No computed style says what colour that is.

So it became AC-61, and there are sixty-one criteria now instead of sixty.

What this does not do

Honesty

THE CLAIM

The a11y gate is green

The pixel diff passes

Adversarial review found nothing

All sixty-one criteria pass

THE LIMIT

Automated rules reach maybe 30–40% of real WCAG failures

It saw no change. It has no idea whether a change would be better

Models checking models. Least reliable tier, never the only one

Nothing here has an opinion about whether the design is good

Green means **no known defect**. It has never meant finished.

Where loops actually fail

It edits the test. The gate goes green and nothing was fixed. Forbid it in writing, and check that the file did not change.

It oscillates. Two wrong states, alternating. Usually two criteria that contradict each other. Cap the iterations and read the log.

It keeps going after it is done. Run the verifier *first*, every pass.

It runs out of context mid-task. Put the state in a file.

It reports success without running anything. Only believe the exit code.

It hangs. Not failing, just waiting. Every step needs a deadline — this project hung three times: six minutes, seven, and an hour.

Take this to work on Monday

Do not start with a whole landing page. Start with one gate.

1. Pick the check your team already does by hand and resents.
2. Make it a program that exits 0 or 1.
3. Put the failure output somewhere an agent can read.
4. Only then put an agent in front of it.

The loop is the easy part. The oracle is the work.

// EVERYTHING FROM TODAY

TURBINE

waysconf.szymonpaluch.com

The eight prompts, in English and Polish · the `.fig` · these slides
and the repository, with the checker and the evidence: `github.com/hculap/waysconf-2026-agentic-loops`

Questions. And then go and put your URL on the board.